Software Architecture

SE6103

Sachin Tharaka
(BSc Honours in Software Engineering)

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: Introduction to Software Architectural pattern
- Topic 03: ADL
- Topic 04: 4+1 Architecture
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: Introduction to Software Architectural pattern
- **Topic 03: ADL**
- Topic 04: 4+1 Architecture
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

Architecture Description Languages (ADL)

Topics Covered

- Motivation for ADL
- Architecture description concepts
- Components, connectors, and configurations
- ADL examples
- Benefits and limitations

Why Describe Software Architecture?

Problems without proper architectural description:

- Miscommunication between developers
- Poor documentation
- Difficult system analysis
- Hard to evaluate architecture quality

Architectural description helps:

- Understand system structure
- Communicate design decisions
- Analyze quality attributes
- Support system evolution

ADL

The source code is too detailed, and informal diagrams are often ambiguous. ADL provides structured and analyzable architectural descriptions.

What is an Architecture Description Language?

A formal language used to represent software architecture in terms of components, connectors, and system configurations.

ADL describes how system parts interact rather than how they are implemented.

- High level abstraction
- Formal syntax
- Tool support for analysis

ADL focuses on structure and interactions, not algorithm implementation.

Architecture Representation

Software architecture usually includes three elements

- Components
- Connectors
- Configurations

Components

A component represents a computational unit in the system.

Examples

- Services
- Databases
- Modules
- UI elements
- Microservices

Characteristics

- Encapsulated functionality
- Interfaces (input/output ports)
- Internal implementation hidden

Connectors

A connector defines communication between components.

Examples

- Function calls
- Message queues
- REST APIs
- Event streams
- Shared databases

Connector responsibilities

- Data exchange
- Control flow
- Synchronization

Configuration

The arrangement of components and connectors in the system.

- System topology
- Component relationships
- Communication structure

Client → API Gateway → Microservices → Database

Key Features of ADL

- Explicit component modeling
- Explicit connector modeling
- Architectural configuration
- Interface definitions
- Behavioral specifications
- Architectural constraints

ADL allows architects to define rules and restrictions within architecture.

Goals of ADL

ADLs aim to support

- Architecture documentation
- System analysis
- Architecture validation
- Performance evaluation
- Early error detection

ADL helps detect architectural problems before implementation begins.

Why ADL is Important

Benefits of using ADL

- Improved communication
- Formal architecture documentation
- Architecture validation and verification
- Reuse of architectural designs
- Early performance analysis

many failures happen because architecture decisions are not clearly documented.

ADL vs Programming Languages

Programming Language

- Implementation focused
- Algorithms and logic
- Detailed code

ADL

- Structure focused
- High level abstraction
- System organization

ADL vs UML

UML

- Semi-formal modeling language
- Visual diagrams
- Widely used in industry

ADL

- Formal architecture specification
- Precise semantics
- Supports analysis tools

UML can represent architecture but ADL is more precise and analyzable.

Types of ADL

Main categories

- Structural ADL
- Behavioral ADL
- Domain specific ADL

Types of ADL

Main categories

- Structural ADL
- Behavioral ADL
- Domain specific ADL

Examples

- Acme
- Wright
- Rapide
- Darwin
- AADL

Example ADL: Acme

Acme is a widely used ADL designed for architecture interchange.

Features

- Simple component connector model
- Extensible
- Used for architecture documentation

Used for

- Tool integration
- Architecture modeling

Example ADL: Wright

Wright is a formal ADL based on process algebra.

Main focus

- Component behavior
- Interaction protocols
- Formal verification

Advantages

- Detect communication errors
- Verify interaction correctness

Example ADL: Rapide

Rapide focuses on event-based architecture modeling.

Key concepts

- Event simulation
- Architecture execution
- Behavior analysis

Used for

- Simulation of system architecture

Example ADL: Darwin

Darwin is used for distributed systems architecture.

Features

- Distributed component modeling
- Dynamic system configuration
- Component interaction specification

Example ADL: AADL

- **AADL stands for Architecture Analysis and Design Language**

Used in

- Aerospace systems
- Embedded systems
- Safety-critical systems

Advantages

- Performance analysis
- Reliability analysis
- Real time system modeling

ADL Tool Support

ADL tools help analyze architecture.

Examples of capabilities

- Architecture simulation
- Performance analysis
- Deadlock detection
- Architectural validation

Benefits

- Detect design flaws early
- Improve architecture quality

ADL Example Architecture

Example: Online Shopping System

Components

- User Interface
- Product Service
- Order Service
- Payment Service
- Database

Connectors

- REST API
- Message Queue
- Database connection

Limitations of ADL

- Limited industrial adoption
- Complex learning curve
- Tool support is not always mature
- Developers prefer UML diagrams

ADL and Modern Architectures

ADL concepts still apply to modern architectures

- Microservices architecture
- Cloud systems
- Event driven systems
- Service oriented architecture

Summary

Architecture Description Languages allow architects to

- Formally describe software architecture
- Define components and connectors
- Analyze architectural properties
- Improve communication among developers

Discussion

Why is it difficult to rely only on source code to understand system architecture?

Key Takeaways

Consider a Food Delivery System.

- Components
- Connectors
- System configuration

Architecture Description Languages (ADL)

Thank You!

Aspect	UML Diagram	ADL Diagram
Purpose	General software modeling	Architecture specification
Representation	Visual diagrams	Formal architecture models
Structure	Components & dependencies	Components, connectors,
Formality	Semi-formal	More formal
Tool support	Very strong	Limited but specialized
Industry usage	Very high	Mostly academic / specialized systems

```
system LoginApp {

    // Components
    component User
    component LoginSystem
    component Database

    // Connectors
    connector Request
    connector Query

    // Configuration (Connections)
    User -> Request -> LoginSystem
    LoginSystem -> Query -> Database
}
```

```
system OnlineShoppingSystem {
    components:
        UI: UserInterface,
        OP: OrderProcessor,
        PG: PaymentGateway,
        DB: Database;

    connectors:
        C1: HTTPConnector,
        C2: HTTPConnector,
        C3: DBConnector;

    attachments:
        UI.userRequest -> C1.client;
        C1.server -> OP.receiveOrder;

        OP.processOrder -> C2.client;
        C2.server -> PG.paymentRequest;

        OP.processOrder -> C3.requester;
        C3.provider -> DB.query;
}
```